

Instacart Image-Based Reorder Prediction Checkpoint

Compiled writeup based on the uploaded project sections, notebooks, and checkpoint artifacts.

Data Set and Image Sources

Our dataset combines behavioral target variables from the Instacart Market Basket Analysis dataset with visual product data from an external product-image source. The practical image source used for this checkpoint is Open Food Facts, which provides publicly accessible product metadata and packaging images through its API and image infrastructure. This allows Instacart-derived reorder behavior to be linked to product images without relying on retailer-specific media pipelines. In the current implementation, products are matched to Open Food Facts entries using product-name search and a string-similarity heuristic, with the highest-scoring acceptable candidate retained for each product. This makes the checkpoint feasible even when barcode-level matching is unavailable, while still preserving a path toward cleaner future joins through UPC or barcode fields if they can be obtained later.

Using a public image source also makes downstream preprocessing easier. Rather than depending on a shopping platform's own display conventions, the image collection stage can remain simple and reproducible, while any optional background cleanup or standardization can be handled later in preprocessing. For the checkpoint subset, this matching procedure produced a small but usable image-labeled dataset suitable for initial model comparisons.

Pipeline

The proposed end-to-end pipeline begins by constructing a product-level dataset in which each observation contains both a product image and a behavioral target derived from Instacart reorder activity. Images are screened for validity, converted to RGB, resized to a fixed input size, and then routed into a unified training pipeline that supports preprocessing, prediction, and evaluation across multiple vision-model families. The checkpoint frames the task as binary classification rather than regression by assigning products to high-reorder and low-reorder categories using thresholded reorder ratios, which makes the experimental comparison easier to interpret at this stage.

This structure also allows custom architectures and transfer-learning baselines to be studied under the same evaluation framework. The project therefore does not treat the custom CNN, ResNet-18, and ViT-based models as disconnected experiments; instead, they are all placed inside one coherent workflow so the team can compare learning from scratch, frozen pretrained features, and fine-tuning within a single checkpoint design.

Preprocessing and Augmentation

For the checkpoint, products in the top quartile of reorder ratio are labeled as the positive class and products in the bottom quartile are labeled as the negative class, while the middle 50 percent are excluded to reduce label ambiguity. After labels are assigned, candidate images are checked for usability and converted into a consistent three-channel RGB format. The intended preprocessing design also includes resizing to 224×224 , tensor conversion, normalization, and stratified splitting into training, validation, and test partitions.

The original project writeup proposed conservative training-time augmentation such as random horizontal flips, slight rotations, and mild brightness or contrast jitter. However, the implementation reflected in the uploaded training notebook is more restrained at this stage: the actual transform pipeline used resizing, tensor conversion, and ImageNet-style normalization, but did not yet apply the planned augmentation operators. In other words, augmentation remains part of the design direction, but the current checkpoint experiments should be understood as baseline runs performed primarily with standardized preprocessing rather than a fully augmented training regime.

Transfer Learning and Architectural Considerations

The modeling pipeline supports both convolutional and transformer-based image classifiers using PyTorch and torchvision. A small custom CNN provides the team-developed baseline, while ResNet-18 serves as a standard convolutional benchmark under three regimes: training from scratch, pretrained-frozen transfer learning, and pretrained fine-tuning. Vision Transformer models are incorporated in the same spirit through pretrained frozen and pretrained fine-tuned variants. This architecture choice keeps the comparison methodologically clean because all models can be trained and evaluated within the same ecosystem and with closely matched training logic.

This design is especially useful for a small checkpoint dataset. When the matched image set is limited, pretrained backbones can provide a more realistic starting point than fully learned deep models, while still allowing the team to measure whether fine-tuning improves on frozen-feature extraction. At the same time, including a custom CNN preserves the requirement that at least one model family be genuinely developed by the team rather than borrowed wholesale from standard libraries.

Experimental Results and Analysis

The uploaded notebook reports six initial model configurations evaluated on a small matched subset of 135 products, split into 94 training examples, 20 validation examples, and 21 test examples. Because the sample is small and the image-matching stage is heuristic, these results should be treated as preliminary rather than definitive. Even so, they are useful for understanding the behavior of the different architectural choices.

Model	Accuracy	F1	ROC-AUC
Custom CNN	0.619	0.750	0.620
ResNet-18 (scratch)	0.524	0.688	0.389
ResNet-18 (pretrained, frozen)	0.571	0.690	0.528
ResNet-18 (pretrained, fine-tuned)	0.619	0.500	0.667
ViT-B/16 (pretrained, frozen)	0.619	0.733	0.537
ViT-B/16 (pretrained, fine-tuned)	0.571	0.710	0.579

The custom CNN performed competitively relative to the larger pretrained models and achieved the strongest F1 score among the reported runs. This is notable because it suggests that, on a very small dataset, a simpler architecture may already be sufficient to capture some coarse visual distinctions associated with high- versus low-reorder products.

The ResNet-18 results were mixed. Training from scratch underperformed on ROC-AUC, which is not surprising given the limited sample size. The pretrained frozen version improved calibration somewhat, while the fine-tuned version produced the strongest ROC-AUC among the ResNet conditions but at the cost of a lower F1 score. This pattern suggests that threshold behavior and class-balance effects may still be unstable.

The ViT variants were promising but not clearly dominant. The frozen ViT achieved a strong F1 score, while the fine-tuned ViT remained competitive without surpassing the simpler CNN baseline on this checkpoint. Taken together, these results support the idea that transfer learning is worthwhile, but they also indicate that the dataset size and image-match quality are currently major bottlenecks.

Conclusion

Overall, the checkpoint shows that the project pipeline is viable: Instacart reorder behavior can be transformed into a binary visual prediction task, external product images can be matched at workable scale, and both CNN-based and transformer-based models can be trained and compared within one experimental framework. The current results should be read as baseline evidence rather than final performance claims, since the matched dataset is still small and the implemented preprocessing is more complete than the implemented augmentation. Even so, the experiments already justify the core project direction. The next most valuable improvements are to expand the matched image set, tighten product-image matching, add the planned light augmentation pipeline, and rerun the same model families under a larger and more stable evaluation setting.

Appendix: Workflow Summary

```
Instacart tabular data + Open Food Facts images
↓
product matching / label creation
↓
image screening and RGB conversion
↓
resize to 224 × 224 and normalize
↓
train / validation / test split
↓
custom CNN, ResNet-18, and ViT comparisons
↓
accuracy, F1, ROC-AUC, confusion matrix
```